

PSTAT 10 Data Science Principles

Lecture 5: Algorithms

John Robin Inston 

johninston@ucsb.edu

University of California, Santa Barbara

August 10, 2026

Introduction

Review: Lecture 4

👉 Last lecture we explored some automation methods such as:

- **Functions**
- Branching (`if`, `else`, `else if`)
- Looping:
 - For Loops
 - While Loops
 - Repeat & Break Loops
- Control Flow

👁️👁️ Outline: Lecture 5

👉 This lecture we will shift our focus to problem solving, including:

- **Algorithmic Thinking**
- Branching Logic
- Iterative Logic
- Recursive Logic

Algorithms

Data Science

What is Data Science?

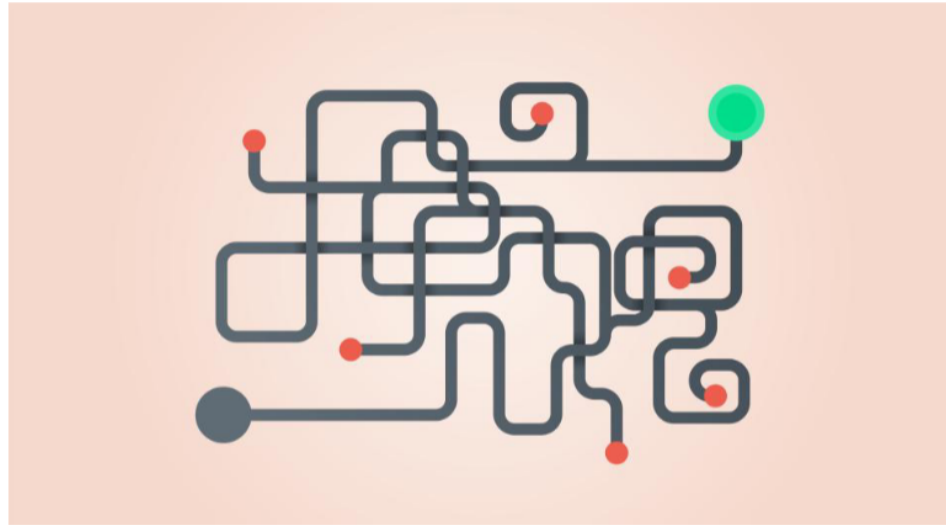
- **Data science** is a combination of subjects including:
 - **+ Math**
 -  **Statistics**
 -  **Programming**
 -  **Computer Science**
 -  **Machine Learning**

Data All Around Us

- **Data** is the single most prolific commodity in the world with over **1 trillion GB being generated per day**.
- This is far too much for any human to handle, so we are required to develop automated procedures using **algorithms**.



What is an Algorithm?



Algorithm Visualization

Algorithm

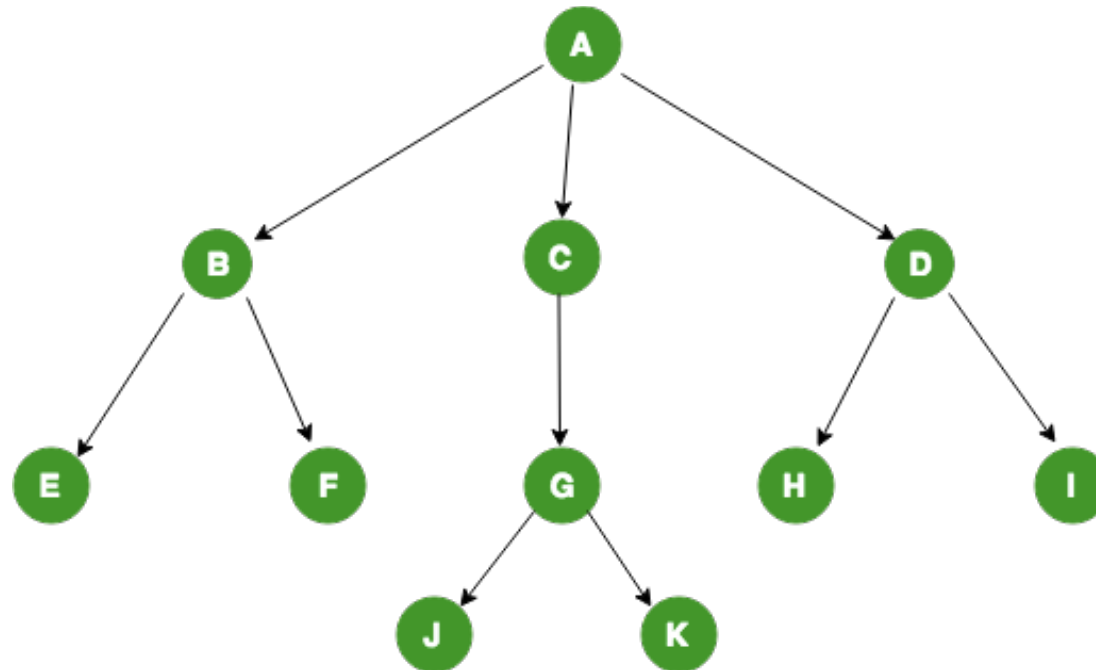
An **algorithm** is a **process / set of rules** performed **sequentially** to solve some kind of problem.

- The rules are often **mathematical** but can be thought of as general functions.
- We have all used algorithms before, particularly in math classes whenever we learned to solve a problem in several steps: calculus and integral rules, long division, algebra etc.

Branching Algorithms

Branching Algorithm

Branching algorithms can be visualized as a case-dependent tree structure, where each branch determines which steps the algorithm applies.



Branching Algorithm

Example — Branching Algorithm

Checking the length of a character string

- Here is a simple example of a branching algorithm that checks that the length of a character string is greater than 5 characters.
- If the string is greater than 5 characters, the algorithm returns **TRUE**, otherwise it returns **FALSE**.

```
1 # Define the algorithm
2 check_string_length <- function(string) {
3   if (nchar(string) > 5) {
4     return(TRUE)
5   } else {
6     return(FALSE)
7   }
8 }
9 # Test the algorithm
10 check_string_length("Hello world!")
```

[1] TRUE

- For more complicated logic, we can use **nested branching** to check **multiple conditions** in a single algorithm.



Nested Branching Statements

Branching statements can be nested inside one another — an `if` / `else if` / `else` block can contain another complete branching block.

- This is useful when a decision **depends on more than one condition**, checked one at a time.
- The general structure looks like:

```
1 if (logical1) {  
2   if (logical2) {  
3     ACTION A  
4   } else {  
5     ACTION B  
6   }  
7 } else {  
8   ACTION C  
9 }
```

Syntax Tips

- **Indent** each nested level further than the block containing it — this makes the structure easy to read at a glance.
- Every opening brace `{` needs a matching closing brace `}` — **close the inner block before the outer one**.
- RStudio auto-indents and highlights matching braces for you — use this to check your work!



Example — Nested Branching

Suppose we want to classify a temperature, but “hot” vs “cool” means something different depending on whether it’s a weekday or a weekend:

```
1 classify_day <- function(temp, weekday) {  
2   if (weekday) {  
3     if (temp > 75) {  
4       return("Hot weekday")  
5     } else {  
6       return("Cool weekday")  
7     }  
8   } else {  
9     if (temp > 85) {  
10      return("Hot weekend")  
11    } else {  
12      return("Cool weekend")  
13    }  
14  }  
15 }  
16 classify_day(80, TRUE)
```

```
[1] "Hot weekday"
```



Notice the **outer branch** picks weekday vs weekend, and the **inner branch** then checks the temperature within it — exactly the pattern you’ll need for the next exercise!

Exercise — Nested Branching Practice

Write a function `check_number` that takes a single numeric input `x` and uses **nested branching** as follows:

1. First check whether `x` is **positive** or **non-positive** (zero or negative).
2. If `x` is positive, nest a second branch that checks whether `x` is **even or odd**, returning `"positive even"` or `"positive odd"`.
3. If `x` is non-positive, nest a second branch that checks whether `x` is **zero or negative**, returning `"zero"` or `"negative"`.

Hint: the modulo operator `%%` is helpful for checking even / odd — recall `x %% 2 == 0` when `x` is even.

04:00

Solution — Nested Branching Practice

```
1 check_number <- function(x) {  
2   if (x > 0) {  
3     if (x %% 2 == 0) {  
4       return("positive even")  
5     } else {  
6       return("positive odd")  
7     }  
8   } else {  
9     if (x == 0) {  
10      return("zero")  
11    } else {  
12      return("negative")  
13    }  
14  }  
15 }
```

```
1 check_number(4)  
2 check_number(-7)  
3 check_number(0)
```

[1] "positive even"

[1] "negative"

[1] "zero"

 Notice the **outer branch** (positive vs. non-positive) is fully resolved — including both its nested branches closed off with `}` — before the `else` for the outer branch even appears.

Exercise — Branching Algorithms (Word Search)

Consider the following `lang_matrix`:

```
1 lang_matrix <- rbind(  
2   c("hello", "goodbye", "friend"),  
3   c("hola", "adios", "amigo"),  
4   c("bonjour", "au revoir", "ami")  
5 )  
6 print(lang_matrix)
```

	[,1]	[,2]	[,3]
[1,]	"hello"	"goodbye"	"friend"
[2,]	"hola"	"adios"	"amigo"
[3,]	"bonjour"	"au revoir"	"ami"

1. Define this matrix.
2. Write a function `word_search` with two character string inputs: **language** (`english`, `spanish` or `french`) and **type** (`greeting`, `farewell` or `friend`).
3. The function should return the cell corresponding to the language and greeting term.

For example:

```
1 word_search("french", "greeting")
```

```
[1] "bonjour"
```

05:00



Solution — Branching Algorithms (Word Search)

An **inefficient first attempt**:

```
1 word_search <- function(a, b) {  
2   if (a == "english" | a == "e") {  
3     if (b == "greeting") {return(lang_matrix[1, 1])}  
4     else if (b == "farewell") {return(lang_matrix[1, 2])}  
5   }  
6   else if (a == "spanish" | a == "s") {  
7     if (b == "greeting") {return(lang_matrix[2, 1])}  
8     else if (b == "farewell") {return(lang_matrix[2, 2])}  
9   }  
10  else if (a == "french" | a == "f") {  
11    if (b == "greeting") {return(lang_matrix[3, 1])}  
12    else if (b == "farewell") {return(lang_matrix[3, 2])}  
13  }  
14  else{ print("ERROR: word not found")}  
15 }
```

Solution — A Better Approach

To demonstrate that there are various ways to accomplish the same task we introduce the function `%in%`, which tests whether a **value is in a vector**.

```
1 3 %in% c(1, 2, 3, 4, 5) # TRUE
```

```
[1] TRUE
```

```
1 6 %in% c(1, 2, 3, 4, 5) # FALSE
```

```
[1] FALSE
```

```
1 # more efficient solution using %in%
2 word_search <- function(language, type) {
3   # Name the matrix indices
4   dimnames(lang_matrix) <- list(c("english", "spanish", "french"),
5                                 c("greeting", "farewell", "friend"))
6   # Use named indices
7   if(
8     language %in% c("english", "spanish", "french") &
9     type %in% c("greeting", "farewell")
10  ) {
11    return(lang_matrix[language, type])
12  } else {
13    print("Error: Word not found!")
14  }
15 }
```

Iterative Algorithms

Inefficiency of Branching Algorithms

Branching algorithms are often excellent choices but they come with some **limitations**:

- Larger numbers of branches are **time consuming to program** manually.
- **Inefficient** for traversing objects of indeterminate length.

```
1 # Example of inefficient branching algorithm
2 bad_in_vector <- function(v) {
3   if (v[1] == 1) {print("1 is in the vector")}
4   else if (v[2] == 1) {print("1 is in the vector")}
5   else if (v[3] == 1) {print("1 is in the vector")}
6   else if (v[4] == 1) {print("1 is in the vector")}
7   else if (v[5] == 1) {print("1 is in the vector")}
8   else if (v[6] == 1) {print("1 is in the vector")}
9   else if (v[7] == 1) {print("1 is in the vector")}
10  else {print("1 is not in the vector")}
11 }
```

What are Iterative Algorithms?

Iterative Algorithm

An **iterative algorithm** is a process that repeats a set of instructions until a specific condition is met.

Example — Iterative Algorithm

Suppose we want to count how many negative numbers appear in a vector. An iterative algorithm **repeats the same check** for every element:

```
1 count_negatives <- function(v) {  
2   count <- 0  
3   for (i in 1:length(v)) {  
4     if (v[i] < 0) {  
5       count <- count + 1  
6     }  
7   }  
8   return(count)  
9 }  
10 count_negatives(c(3, -2, -7, 5, -1))
```

```
[1] 3
```

 Notice the pattern: **loop over every index**, **check a condition**, then **update something** — this is exactly the structure you'll use in the next exercise!

Exercise — Iterative Algorithms

Write a function `alter_entry` that:

1. Takes a vector (of any data type) as input, and
2. Returns the same vector, but replacing every instance of the number `7` in the vector with `0`.

Below are some examples of the function output:

```
1 alter_entry(c(7,2,3,7,1))  
2 alter_entry(c(10, 7, 3, 7, 8))
```

```
[1] 0 2 3 0 1
```

```
[1] 10 0 3 0 8
```

03:00

Solution — Iterative Algorithms

1. First we construct the surrounding function.
2. Then we define the for loop iterating over the indices of the input vector.
3. Each iteration checks whether the value at the current index is equal to `7`. If it is, we replace it with `0`.

```
1 # define alter_entry function
2 alter_entry <- function(v) {
3   for(i in 1:length(v)) {
4     if(v[i] == 7){
5       v[i] <- 0
6     }
7   }
8   return(v)
9 }
```

```
1 alter_entry(c(7,2,3,7,1))
2 alter_entry(c(10, 7, 3, 7, 8))
```

```
[1] 0 2 3 0 1
```

```
[1] 10 0 3 0 8
```

- Remember you should still always try to avoid looping when possible.
 - Lets see if we can improve the efficiency of our previous function by introducing the `which()` function.

The `which()` Function

A very helpful function!

- Often we would like to return the indices of the elements of a vector that satisfy a certain condition.
 - For this we can use the `which()` function.
 - The `which()` function takes a logical vector as input and returns the indices of the `TRUE` values.
 - It can be helpful for avoiding iterative logical queries!
- An alternative solution to our previous example using the `which()` function could be:

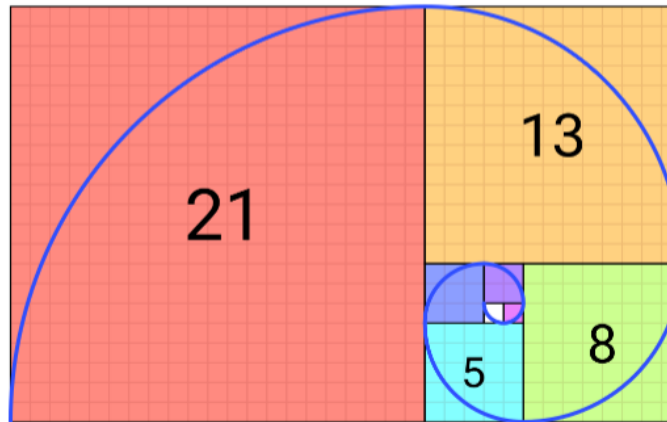
```
1 alter_entry <- function(v) {  
2   v[which(v == 7)] <- 0  
3   return(v)  
4 }
```

 Which function is more efficient?

Fibonacci Sequence

In preparation for the next exercise we briefly introduce the **Fibonacci sequence**.

The Fibonacci sequence is a mathematical pattern occurring commonly in nature. It appears in the golden ratio, which is found in petals, seashells, and galaxy spirals.



Golden Ratio

The sequence itself is constructed by starting with the **initial values** 0 and 1, and every subsequent value is the **sum of the previous two**, i.e.

$$x_n = x_{n-1} + x_{n-2}; \quad x_1 = 0, \quad x_2 = 1.$$

Exercise — Fibonacci Sequence

Write an iterative function `fib_n` that returns the n -th term of the Fibonacci sequence:

1. The function should take **one numerical scalar input** `n`.
2. The function should **return a numeric value** representing the n -th term of the Fibonacci sequence.
3. **Ignore exception cases** and do not worry about robustness — assume the input will always be a positive integer.

```
1 fib_n(10)
2 fib_n(25)
```

```
[1] 34
```

```
[1] 46368
```

05:00

Solution — Fibonacci Sequence

1. We start once again by defining the function and its input.
2. Next we define the base cases for the first two terms of the Fibonacci sequence.
3. Finally we define a for loop that iteratively calculates the Fibonacci sequence up to the n -th term.

```
1 # define fib_n function
2 fib_n <- function(n){
3   if(n == 1) {
4     return(0)
5   } else if (n==2) {
6     return(1)
7   } else {
8     temp <- c(0,1)
9     for(i in 3:n){
10      temp[i] <- temp[i-1] + temp[i-2]
11    }
12    return(temp[n])
13  }
14 }
```

```
1 fib_n(10)
2 fib_n(25)
```

[1] 34

[1] 46368

Combining Everything

Notice that in our previous example we were required to use both **branching** and **iteration**. It is common for algorithms to require the use of several tools.

- To understand and eventually design algorithms it is important to know how the individual pieces / steps work and what they are doing.

Certain tricks come up more often than others, such as:

- **Iterating through vector indices** (**enumeration**);
- Storing and updating **temporary** or **intermediate** values; and
- **Utilizing data structure** and specific behavior.

Let's see if we can combine these tools to solve a problem in the next exercise!

Exercise — Last Negative

Combine some of the tools we have been studying by defining a function called `last_negative` that:

1. Takes a numeric vector `v`.
2. Returns the last (index-wise) negative value in `v` and its index (in a 2-element vector).
3. If `v` does not have any negative values, returns the character string `"No negative values"`.

Below are some examples of the function output:

```
1 last_negative(c(-1,2,6,-2,8,-9))  
2 last_negative(c(5,2,1,4,5,6,9))
```

```
[1] -9 6
```

```
[1] "No negative values"
```

05:00

Solution — Last Negative

1. We start by defining the function and its input.
2. Next we define a temporary variable `temp` to store the last negative value and its index, or a message if no negative values are found.
3. Finally we define a for loop that iterates through the vector, checking for negative values and updating `temp` accordingly.

```
1 last_negative <- function(v) {  
2  
3   temp <- "No negative values" # storing temporary vector  
4  
5   for(i in 1:length(v)){  
6     if(v[i] < 0) {  
7       temp <- c(v[i], i)  
8     }  
9   }  
10  return(temp)  
11 }
```

```
1 last_negative(c(-1,2,6,-2,8,-9))  
2 last_negative(c(5,2,1,4,5,6,9))
```

```
[1] -9 6
```

```
[1] "No negative values"
```

Nested Loops



Nested Loops

Sometimes we may wish to run **loops inside other loops** (e.g. iterating over first the rows and then the columns of a matrix), and to do so we use **nested loops**.

Example Nested For Loop

```
1 for(LOGICAL1, often index-based) {  
2   DO OPERATION1  
3  
4   for(LOGICAL2, often index-based){  
5     DO OPERATION2  
6   }  
7 }
```

- Nested loops are useful but **slow**!
- Good practice is to **only use them when necessary**, prioritising vectorized solutions.

Exercise — Two Sum

Write a function called `two_sum` that:

1. Takes an **input vector** `v` and a **numerical scalar target**, and
2. Returns the indices of the two entries of `v` that **add up to the target**.

Assume that:

- Two indices cannot be equal (i.e. `target` will not be twice any value in `v`);
- If no values summing to `target` are found, return `NULL`; and
- Only one such pair of elements in `v` sum to the `target`.

```
1 two_sum(c(5,6,0,1,4),11)
```

```
[1] 1 2
```

05:00

Solution — Two Sum

```
1 # define two_sum function
2 two_sum <- function(v, target) {
3   n <- length(v)
4   for (i in 1:(n - 1)) {
5     for (j in (i + 1):n) {
6       if(v[i] + v[j] == target) {
7         return(c(i, j))
8       }
9     }
10  }
11  return(NULL)
12 }
```

 See whether you can design a more efficient function using vectors!

Searching & Sorting

Searching & Sorting

Famous Problems

Two very famous problems in computer science are:

1. How to **search data structures for specific values**; and
2. How to **sort unsorted numerical vectors**.

R has **built in functions** to handle both of these problems (i.e. `sort()` and `%in%`) but to understand the challenges of these problems we take a look at two specific solutions:

1. **Bubble sorting**
2. (Iterative) **binary searching**

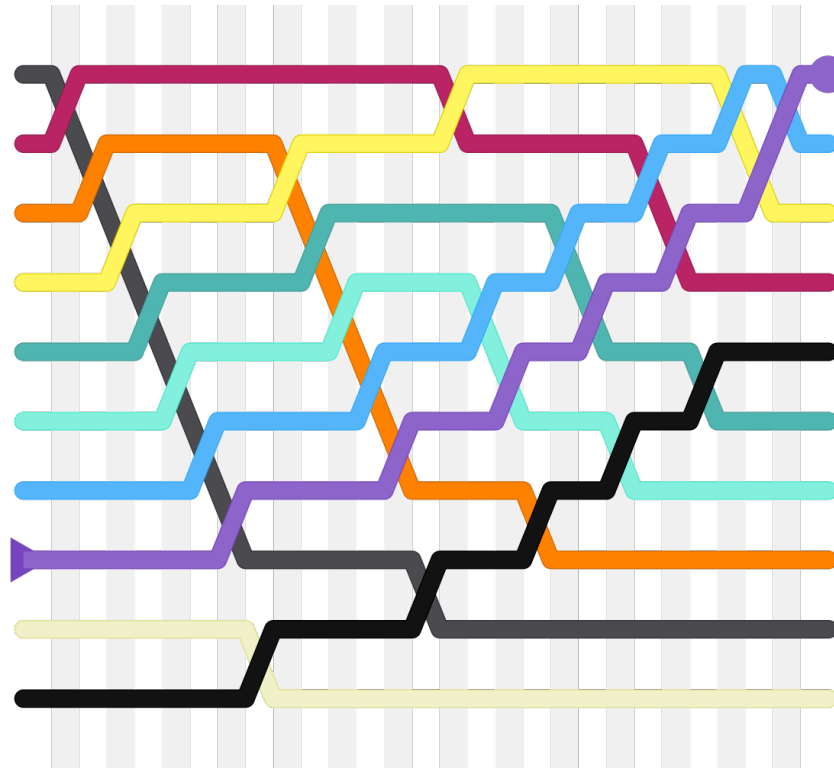


Sorting.

Bubble Sort

In 2007, former Google CEO Eric Schmidt asked then presidential candidate Barack Obama during an interview about the best way to sort one million integers.

Obama paused for a moment and replied *“I think the bubble sort would be the wrong way to go.”*



Bubble Sorting Visualization

The Bubble Sort Algorithm

Main Idea

- Assume our input is a (potentially long) vector v of unsorted values.
- The idea is that we let the “lighter” values “sink” to the bottom.
- Apply the following algorithm:
 - Iterate through the entire vector.
 - For every pair of indices j and $j+1$, if $v[j] > v[j+1]$ swap the values at those indices.
 - Stop iterating when the vector is fully sorted.
 - The desired output is a vector containing the sorted values of v .

Example — Defining Bubble Sort

Implementation

```
1 bubble_sort <- function(v) {  
2   n <- length(v)  
3   for (i in 1:(n - 1)) { # iterate through n-1 loops  
4     is_swapped <- FALSE  
5     for (j in 1:(n - i)) { # swap values at specific indices  
6       if (v[j] > v[j + 1]) {  
7         temp <- v[j]  
8         v[j] <- v[j + 1]  
9         v[j + 1] <- temp  
10      is_swapped <- TRUE  
11    }  
12  }  
13  if (!is_swapped) { # break out of loop if fully sorted  
14    break  
15  }  
16 }  
17 return(v)  
18 }
```

Testing the Algorithm

As an **example of our algorithm in action** let's define a vector of 30 numbers in a random order:

```
1 bubble_sort_test <- sample(1:100, 30, replace = FALSE)  
2 bubble_sort_test
```

```
[1] 46  6 78 100 94 36 65 84 10 72 35 13 91 69 88 23  
37 57 63  
[20] 99 31 41 48 50 32 61 29  5 56 93
```

We apply the bubble sort algorithm:

```
1 bubble_sort(bubble_sort_test)
```

```
[1]  5  6 10 13 23 29 31 32 35 36 37 41 46 48 50 56  
57 61 63  
[20] 65 69 72 78 84 88 91 93 94 99 100
```

Binary Search

Idea behind Binary Search

- Assume that I have picked a number between 1 and 1024 and tell you to guess which number I have chosen.
 - You can only use *higher* or *lower* as clues.
 - You have to guess the correct number in 11 or fewer attempts.
- Can you design an algorithm that always guesses correctly?

Binary Search Structure

The general structure of the binary search algorithm is as follows:

- Begin with a sorted vector `v` and a target value `target`.
- If the midpoint of `v` equals the `target`, end.
 - If the midpoint is greater (less) than `target`, repeat the previous step with the midpoint of the upper (lower) half of the vector.
- Repeat until `target` is found.

Binary Search Algorithm

Implementation

```
1 # define binary search algorithm
2 binary_search <- function(v, target) {
3   left <- 1
4   right <- length(v)
5   while (left <= right) {
6     mid <- floor((left + right) / 2) # check the vector midpoint
7     if (v[mid] == target) {
8       return(mid)
9     } else if (v[mid] < target) { # check lower
10      left <- mid + 1
11    } else {
12      right <- mid - 1
13    }
14  }
15  return(NULL)
16 }
```

Binary Search Example

```
1 # define a sorted vector
2 test_vector <- c(1, 3, 5, 7, 9, 11, 13, 15, 17, 19)
3 binary_search(test_vector, 7) # returns 4
```

```
[1] 4
```

Recursion

Recursion is another common algorithmic pattern.

- It is the act of defining a problem in terms of simpler versions of itself.
- Recursion consists of a **base case** and a **recursive step**.

```
1 # recursion general form
2 recursion <- function(INPUTS) {
3   if (TERMINATION CONDITION) { # base case
4     END RECURSIVE FUNCTION
5   } else {
6     new_inputs <- DO SOMETHING # recursive step
7     recursion(new_inputs)
8   }
9 }
```

Example — Recursive Fibonacci

Recall that we previously wrote an *iterative* Fibonacci algorithm.

- Iterative approaches are effective, and our previous function worked perfectly.
- However, iterative setups can be **complicated to implement**.
- Furthermore, the main drawback of iteration is typically **computational cost and time** — it is often not very efficient!

Let's try using **recursion** to create a new version of the `fib_n` function.

- We have the same setup: take an input n and return the n -th term of the Fibonacci sequence.
- This time, construct the function using recursion.

Recursive Fibonacci — Implementation

```
1 # recursive fibonacci function
2 fib_recur <- function(n) {
3   if (n <= 2) { # base case
4     return(n-1)
5   }
6   return(fib_recur(n - 1) + fib_recur(n - 2)) # recursive step
7 }
8 # example output
9 fib_recur(25)
```

```
[1] 46368
```

 The recursive Fibonacci function works exactly the same, however the implementation is **much faster**. The drawback is that the logic behind the implementation is less intuitive.

Summary

Topics Covered

 Today we began exploring problem solving:

- Algorithmic Thinking
- Branching Logic
- Iterative Logic
- Recursive Logic



Next Class



Next class we look at:

- Storing data
- Factor data